

Informe de Seguridad del Sitio

Controles técnicos activos, cumplimiento OWASP y plan de respuesta ante virus, malware, vulnerabilidades e intrusiones.

Dominio: **www.fandez.cl**

Versión del documento: 1.0 · 19 septiembre 2026

Alcance: aplicación Node.js / Express en Hostinger + MySQL + pagos

Estado al cierre de este informe

Auditoría de dependencias de producción: **0 vulnerabilidades** (`npm audit --omit=dev`). Cabeceras de seguridad con Helmet/CSP activas. Escáner Hostinger/Patchstack debe reflejar cero alertas tras Redeploy.

1. Índice

2. Resumen ejecutivo	3
3. Controles de seguridad activos	3
4. Mapa OWASP Top 10	5
5. Plan de respuesta ante incidentes (qué hacer)	6
6. Contactos y canales	11
7. Checklist mensual de mantenimiento	11
8. Anexos técnicos	12

2. Resumen ejecutivo

Fandez es una plataforma de servicios del hogar con pagos reales, datos personales, documentos KYC y chat en vivo. La seguridad se organiza en tres capas:

- 1. **Prevención** — contraseñas con bcrypt, HTTPS, Helmet/CSP, roles, CSRF en admin, rate limit, panel admin en path secreto, MFA, validación de archivos.
- 2. **Detección** — logs de seguridad, Hostinger Vulnerabilities (Patchstack npm), backups cifrados, alertas SOS / Sofía, correo de soporte.
- 3. **Respuesta** — este playbook: pasos concretos según el tipo de amenaza (malware, cuenta comprometida, defacement, ransomware, DDoS, etc.).

Importante

Ningún sitio es 100 % invulnerable. Este documento te prepara para *actuar rápido*, contener el daño y recuperar el servicio con evidencia. Si hay peligro físico a personas, prioriza **133** (Carabineros) — eso está en el protocolo de seguridad en terreno, no sustituye a este informe técnico.

3. Controles de seguridad activos

3.1 Acceso y autenticación

Control	Qué hace
Roles (client, provider, tecnico, admin)	Cada ruta exige el rol correcto; cuentas desactivadas pierden sesión.
Contraseñas bcrypt	No se guardan en texto plano; comparación segura en login.
Rate limit login	Limita intentos de acceso / recuperación (mitiga fuerza bruta).
Rate limit global	~150 req/min por IP en rutas dinámicas.

Sesión HttpOnly + Secure + SameSite	Cookie fandez.sid no legible por JS; solo HTTPS en producción.
SESSION_SECRET obligatorio	La app no arranca en producción sin secreto fuerte (≥24 caracteres).
Panel admin oculto	Path tipo /ops-... (no /admin); opcional allowlist IP.
CSRF en admin	Tokens por sesión en acciones POST del panel.
MFA admin	Segundo factor TOTP disponible para administradores.

3.2 Red, cabeceras y configuración

Control	Qué hace
HTTPS + HSTS	Tráfico cifrado; fuerza HTTPS en navegadores.
Helmet + CSP	Reduce XSS, clickjacking y carga de scripts no autorizados.
Sin X-Powered-By	No revela Express a atacantes.
Canonical www	Redirección a www.fandez.cl (una sola instancia con BD).
Boot seguro	Valida APP_URL, ADMIN_PASSWORD, webhook MP, etc. al iniciar.
Modo producción	APP_MODE=production — pagos reales, sin atajos demo.

3.3 Datos, archivos y pagos

Control	Qué hace
MySQL parametrizado	Reduce inyección SQL.
Uploads tipados	Solo JPG/PNG/WEBP/PDF; validación por magic bytes; límites de tamaño.
Media autenticada	Fotos de pedidos / KYC no son públicas a ciegas.
Mercado Pago / Transbank	Cobros en pasarela externa; webhook con secreto.
Backups cifrados	Export diario a GitHub (rama backups) con token + clave.
Logs de seguridad	Eventos de acceso / bloqueos en security_logs.

3.4 Producto (seguridad humana en visitas)

Complemento operativo (ver también docs/SEGURIDAD-ALERTAS.md):

- KYC de socios, código de llegada, GPS en visita, chat sin exponer teléfono al Guardián.
- Botón SOS cliente/técnico → incidente + alerta admin + opción llamar 133.
- Sofía detecta lenguaje de amenaza/acoso y escala a soporte.

4. Mapa OWASP Top 10 (cómo cumple Fandez)

OWASP	Riesgo	Mitigación en Fandez
A01 Control de acceso	Ver datos de otro usuario	Roles, ownership en pedidos, admin path secreto, IP allowlist
A02 Fallos criptográficos	Robo de secretos/contraseñas	HTTPS, bcrypt, cookies Secure, secretos en env Hostinger (no en Git)
A03 Inyección	SQL / XSS	Queries parametrizadas, CSP, escape EJS, validación inputs
A04 Diseño inseguro	Flujos de pago abusables	Estados de pedido, cobros adicionales con aprobación, webhooks firmados
A05 Mala configuración	Headers / debug	Helmet, sin X-Powered-By, boot seguro, NODE_ENV=production
A06 Componentes vulnerables	CVE en npm	Auditoría 0 en prod; escáner Hostinger; actualizar deps
A07 Auth fallida	Fuerza bruta / sesión	Rate limit, MFA admin, sesiones fuertes
A08 Integridad	Código alterado	Deploy desde GitHub; no subir ZIP a mano salvo emergencia
A09 Logging	No detectar intrusión	security_logs, correo Hostinger, backups
A10 SSRF	Servidor pide URLs malas	Sin fetch genérico a URLs de usuarios

Nota: “cumplir OWASP” aquí significa **aplicar las prácticas del Top 10**. No es una certificación formal (esa la emite un auditor externo).

5. Plan de respuesta — qué hacer en cada caso

Usa siempre este orden: **1 Contener** → **2 Evidencia** → **3 Erradicar** → **4 Recuperar** → **5 Aprender**.

5.1 Severidad

- S1 CRÍTICO** datos robados, ransomware, defacement, pagos manipulados, admin comprometido
- S2 ALTO** malware en upload, cuenta de socio/cliente hackeada, CVE explotable
- S3 MEDIO** escaneo, spam, DDoS leve, falso positivo Hostinger

Primera hora (cualquier S1)

- No borrar logs ni “arreglar a ciegas” sin capturas.
- Anotar hora (Chile), qué viste, IP si aparece, usuario afectado.
- Si hay filtración de datos personales: preparar aviso a afectados (Ley 19.628 / buenas prácticas) — contactar abogado si el volumen es alto.
- Cambiar SESSION_SECRET y contraseñas admin solo *después* de contener (ver casos abajo).

Caso A — Hostinger avisa vulnerabilidades npm S2

1. Abre hPanel → Security → Vulnerabilities y anota paquetes/CVE.
2. En el Mac: `npm audit --omit=dev` en el repo.
3. Actualiza versiones seguras (sin `--force` a ciegas).
4. Commit + push a main + **Redeploy**.
5. Verifica que el panel Hostinger quede en cero.

Caso B — Intento de fuerza bruta / login masivo S3-S2

1. Revisa `security_logs` en admin y Runtime logs Hostinger.
2. Confirma que rate limit sigue activo (429 en demasiados intentos).
3. Si persiste: activa/ajusta `ADMIN_IP_ALLOWLIST`; cambia contraseña admin; activa MFA.
4. No abras enlaces de “reset” que lleguen por correo sospechoso.

Caso C — Cuenta de cliente/socio/técnico comprometida S2

1. En admin: desactiva la cuenta (`active=false`) — corta sesión.
2. Revisa pedidos recientes, chats y pagos de esa cuenta.
3. Contacta al dueño legítimo por un canal verificado (no solo el email de la cuenta).
4. Tras limpiar: reset de contraseña + reactivación controlada.
5. Si hubo cobros indebidos: abrir disputa en Mercado Pago / banco y documentar.

Caso D — Sospecha de admin hackeado S1

1. Desde Hostinger: reinicia la app o pon mantenimiento si es posible.
2. Rota de inmediato: `SESSION_SECRET`, `ADMIN_PASSWORD`, `ADMIN_PATH` (nuevo path), tokens MP/SMTP, `BACKUP_SYNC_TOKEN`.
3. Redeploy desde GitHub main limpio (no uses archivos raros del servidor).
4. Revisa usuarios admin, MFA, allowlist IP.
5. Audita cambios recientes en BD (precios, promos, usuarios).

Caso E — Virus / malware en archivo subido (foto, boleta, PDF) S2

1. No abras el archivo en tu PC personal sin sandbox.
2. En admin: identifica el pedido/usuario y elimina o aísla el archivo en `uploads`.
3. Desactiva temporalmente al usuario que lo subió si hay mala fe.
4. Verifica que el tipo MIME/extensión siga restringido (JPG/PNG/WEBP/PDF).
5. Si el archivo se ejecutó en algún lado (poco probable en Node sirviendo static): trata como Caso G.

Caso F — Página defaceada / HTML raro / redirects a phishing S1

1. Redeploy limpio desde GitHub (sobrescribe archivos infectados del deploy).
2. Cambia todos los secretos de entorno (ver Caso D).
3. Revisa DNS del dominio en Hostinger (que no apunte a otro lado).
4. Escanea el correo corporativo (SMTP) por reglas de reenvío desconocidas.

5. Avisa a usuarios solo si hubo phishing activo con datos.

Caso G — Código malicioso en el servidor (shell, crypto-miner) S1

1. Redeploy desde GitHub + rotación total de secretos.
2. Revisa procesos / CPU en Hostinger; contacta soporte Hostinger si la carga es anormal.
3. Compara el árbol de archivos del deploy con el repo (no debe haber PHP/shell sueltos).
4. Rota claves MySQL si hay duda de acceso a BD.
5. Restaura desde backup cifrado solo si el código limpio + BD sospechosa.

Caso H — Ransomware / BD cifrada o borrada S1

1. No pagues rescate sin asesoría legal.
2. Detén escrituras si puedes; exporta lo que quede.
3. Recupera el último backup bueno: rama GitHub backups + decrypt-github-backup.js.
4. Redeploy app limpia + restore BD en entorno controlado.
5. Rota todas las claves; investiga vector de entrada.

Caso I — Filtración de datos personales (PII) S1

1. Contén el acceso (Casos C/D/G según origen).
2. Documenta qué datos salieron (emails, RUT, direcciones, documentos KYC).
3. Avisa a afectados con hechos claros (qué, cuándo, qué hacer ellos: cambiar clave).
4. Conserva evidencia (logs, backups) — no la borres.
5. Consulta abogado/compliance si el impacto es masivo.

Caso J — Pagos manipulados / cobros fantasma S1

1. Congela el pedido en admin; no reembolses a ciegas sin ver MP.
2. Revisa panel Mercado Pago / Transbank vs. pedido en Fandez.
3. Verifica MP_WEBHOOK_SECRET y que los webhooks no se estén reenviando a otro lado.
4. Si hay fraude de tarjeta: sigue el proceso del adquirente (contracargo).

Caso K — DDoS / sitio caído por tráfico S2

1. Confirma en Hostinger (CDN hcdn) y /health.
2. Activa/valida rate limit; pide a Hostinger mitigación CDN si aplica.
3. No abras puertos ni desactives HTTPS “para probar”.
4. Cuando vuelva: revisa logs de IPs top y bloquea si es repetitivo.

Caso L — Phishing / virus en el correo del equipo S2

1. No descargues adjuntos ni des permisos OAuth a sitios raros.
2. Cambia contraseña del buzón Hostinger; revisa reenvíos y filtros.
3. Si se filtró SMTP_PASS: rota la clave SMTP en la app y reinicia.
4. Avisa al equipo: el admin real nunca pide la contraseña por WhatsApp.

Caso M — Bug explotado (XSS, IDOR, lógica de negocio) S2-S1

1. Reproduce en staging o local; no ignores reportes de usuarios.
2. Parchea en el repo; tests mínimos; deploy.
3. Revisa si alguien abusó el bug (pedidos/pagos) y revierte efectos.
4. Agradece reportes responsables; no publiques detalles hasta parchear.

Caso N — Compromiso de GitHub S1

1. Revoca tokens/PAT; activa 2FA en GitHub si no está.

2. Revisa commits recientes en main; revertir lo malicioso.
3. Rota secrets de Actions (FANDEZ_BACKUP_*).
4. Redeploy solo desde commit verificado por ti.

Recuperación estándar post-incidente

1. Redeploy limpio desde main.
2. `curl -s https://www.fandez.cl/health → ok: true.`
3. Login admin + prueba de un pedido demo/pago sandbox si aplica.
4. Anotar en un registro interno: fecha, caso, acciones, responsable.
5. Programar mejora (parche, monitoreo, MFA) en la semana siguiente.

6. Contactos y canales

Canal	Uso
hPanel Hostinger	Redeploy, env vars, vulnerabilidades, DNS, correo, tickets soporte
GitHub Mcanetet/fandez-app	Código fuente, Actions de backup, PRs de seguridad
Panel admin Fandez (/ops-...)	Usuarios, reclamos seguridad, logs, precios
Mercado Pago / Transbank	Pagos, contracargos, webhooks
133 Carabineros	Amenaza física en visitas (no es incidente TI)

Guarda fuera de este PDF (gestor de contraseñas): SESSION_SECRET, ADMIN_PASSWORD, path admin, claves MP/SMTP/DB, BACKUP_GITHUB_ENCRYPT_KEY.

7. Checklist mensual de mantenimiento

☐	Tarea
☐	Correr npm audit --omit=dev y dejar en 0
☐	Revisar Hostinger → Vulnerabilities
☐	Verificar backup diario en rama backups
☐	Revisar security_logs / reclamos tipo seguridad
☐	Confirmar MFA admin activo y path admin no público
☐	Probar /health y un login
☐	Rotar secretos si hubo salida de alguien del equipo

8. Anexos técnicos

8.1 Comandos útiles

```
# Salud
curl -s https://www.fandez.cl/health | jq .

# Auditoría dependencias producción
npm audit --omit=dev

# Descifrar backup (en tu Mac)
BACKUP_GITHUB_ENCRYPT_KEY='...' node scripts/decrypt-github-backup.js ./archivo.enc ./snapshot.json
```

8.2 Documentos internos relacionados

- docs/SEGURIDAD-OWASP.md — mapa OWASP y npm audit
- docs/SEGURIDAD-ALERTAS.md — SOS / seguridad en terreno
- docs/BACKUP-GITHUB.md — backups cifrados
- HOSTINGER.md — deploy y variables de entorno

8.3 Archivos de código relevantes

- middleware/security.js — Helmet, CSP, rate limits
- middleware/csrf.js · middleware/auth.js
- lib/password.js · lib/mfa.js · lib/appMode.js
- lib/uploads.js · routes/media.js
- scripts/disaster-recovery.js

Fandez — Informe de Seguridad v1.0 · Generado para uso interno del operador del sitio.

Este documento no sustituye auditoría legal ni pentest externo. Actualizar tras cada incidente material o cambio mayor de infraestructura.

Clasificación: Confidencial.